GMT (Generic Movie Tickets)

Software Requirements Specification

Version 4

11/12/2026

Group #3
Andres Akerlundh, Madison Stevens,
Haoze Li

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

Revision History

Date	Description	Author	Comments
9/14/26	Version 1.0	Group 3	First Draft
10/8/26	Version 2.0	Group 3	Software Design
10/22/26	Version 3.0	Group 3	Test Plan
11/12/26	Version 4.0	Group 3	Architecture Design

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Dr. Gus Hanna	Software Eng. Instructor, CS 250	

Table of Contents

Revision History..... ii

Document Approval..... ii

1. Introduction..... 1

   1.1 Purpose..... 1
   1.2 Scope..... 1
   1.3 Definitions, Acronyms, and Abbreviations..... 1
   1.4 References..... 1
   1.5 Overview..... 1

2. General Description..... 2

   2.1 Product Perspective..... 2
   2.2 Product Functions..... 2
   2.3 User Characteristics..... 2

2.4 General Constraints..... 2
2.5 Assumptions and Dependencies..... 2

3. Specific Requirements..... 2

3.1 External Interface Requirements..... 3
3.1.1 User Interfaces..... 3
3.1.2 Hardware Interfaces..... 3
3.1.3 Software Interfaces..... 3
3.1.4 Communications Interfaces..... 3
3.2 Functional Requirements..... 3
3.2.1 Employee Operation..... 3
3.2.2 Purchase Tickets..... 3
3.3 Use Cases..... 3
3.3.1 Use Case #1: Log In..... 3
3.3.2 Use Case #2: Purchase Movie Ticket..... 3
3.3.3 Use Case #3: Manage Movies and Screening Info..... 3
3.4 Classes / Objects..... 3
3.4.1 Class / Object #1 Movie..... 3
3.4.2 Class / Object #2 Users..... 3
3.5 Non-Functional Requirements..... 4
3.5.1 Performance..... 4
3.5.2 Reliability..... 4
3.5.3 Availability..... 4
3.5.4 Security..... 4
3.5.5 Maintainability..... 4
3.5.6 Portability..... 4
3.6 Inverse Requirements..... 4
3.7 Design Constraints..... 4
3.8 Logical Database Requirements..... 4
3.9 Other Requirements..... 4

4. Analysis Models..... 4

4.1 Sequence Diagrams..... 5
4.3 Data Flow Diagrams (DFD)..... 5
4.2 State-Transition Diagrams (STD)..... 5

5. Change Management Process..... 5

6. Software Design Specification
6.1 System Description
6.2 Software Architecture Overview
6.2.1 Software Architecture (SWA) Diagram
6.2.2 SWA Description
6.2.3 UML Class Diagram
6.2.4 Class Descriptions

6.2.4.1 Class: Movie
6.2.4.2 Class: Users
6.2.5 Data Structures
6.3 Development Plan and Timeline
6.3.1 Partitioning of Tasks
6.3.2 Timeline
6.3.3 Team Member Responsibilities
7. Test Plan
7.1 Test Objectives and Scope
7.1.1 Testing objective
7.1.2 Testing which function
7.1.3 Unit, Functional/Integration and System testing
7.1.4 Test data, erroneous input and failure scenarios
7.2 Test Cases and Coverage
7.2.1 Test Case Summary
7.2.2 Test Targets and Scope
7.2.3 Coverage of the Functions Listed in 7.1.2
7.3 GitHub Link
8. Architecture Design with Data Management
8.1 Introduction
8.2 Updated Software Architecture Diagram
8.2.1 Data Tier and Application Server Modules
8.2.2 Data-Related Connectors
8.3 Changes from the Initial Design (Assignment 2)
8.4 Data Management Strategy
8.4.1 Data Inventory and Sensitivity
8.4.2 Database Technology: SQL
8.4.3 Number of Databases and Logical Organization
8.4.4 Transactions, Concurrency and Live Data
8.4.5 Protection of Sensitive Data
8.4.6 Backup, Recovery and Retention
8.4.7 Capacity Estimate
8.5 Design Decisions, Alternatives and Tradeoffs
8.5.1 Database Technology
8.5.2 Number and Organization of Databases
8.5.3 Handling of Payment and Credential Data
8.5.4 Accepted Tradeoffs

A. Appendices..... 5

A.1 Appendix 1..... 5
A.2 Appendix 2..... 5

1. Introduction

This Software Requirements Specification describes the requirements for a cinema system. The system enables customers to view movie information, search for showtimes, select seats, and purchase movie tickets, while also allowing authorized cinema staff to manage movies, screenings, and bookings. This document provides an overview of the system and outlines its user requirements, functional requirements, and non-functional requirements, including usability, reliability, performance, and security.

1.1 Purpose

The purpose of this document is to define the requirements for the theater ticketing system, capturing the needs of customers and theater administrators. This SRS was developed through a client requirements interview, followed by a team brainstorming session to identify and refine additional functionality, evaluate feasibility, and document potential issues to guide future updates. This document provides a clear and detailed picture of the system's scope, goals, and expected behavior, including its target users, interface design, and the hardware and software needed to support it. The intended audience includes customers, the development and testing teams, and project stakeholders. They can rely on this document as a shared reference for a consistent understanding of the system's functionality throughout the software development lifecycle, from design through implementation.

1.2 Scope

This document defines the requirements for a movie ticketing website that enables users to browse movie listings, view showtimes, select seats, and book tickets online. It covers the needs of administrative users, customers, and the engineers who will be referring to this document, and describes the functionality required to support ticket booking, show scheduling, and related customer-facing features.

In scope for this system are movie listings and showtime browsing, seat selection and booking, payment processing, email confirmation of bookings, QR code generation for ticket check-in, and other related features supporting the end-to-end ticketing experience.

This SRS specifies the requirements for the software to be developed and can also serve as a reference for evaluating existing or third-party software components that may be used in building the system. It can be used directly to guide development, or adapted as a template for

defining requirements on similar projects. It does not prescribe any particular method, notation, or tool to be used in preparing the specification itself.

1.3 Definitions, Acronyms, and Abbreviations

FAQ	Frequently Asked Questions
CRM	Customer Relationship
SRS	Software Requirements Specification
UI	User Interface
HTTPS	Hypertext Transfer Protocol Secure
API	Application Programming Interface
PCI	Payment Card Industry
SMTP	Simple Mail Transfer Protocol

1.4 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, Institute of Electrical and Electronics Engineers, 1998.
2. Theater Ticketing System Client Requirements Interview, provided as part of course materials, 2026.
3. Software Requirements Specification Template, provided by course instructor, 2026.

1.5 Overview

The remaining sections of this document provide a general description, including characteristics of the users of this project, the product's hardware, and the functional and data requirements of the product. General description of the project is discussed in section 2 of this document. Section 3 gives the functional requirements, data requirements and constraints and assumptions made while designing the Theater Ticketing System. It also gives the user a viewpoint of the product. Section 3 also gives the specific requirements of the product, and

discusses the external interface requirements and gives detailed description of functional requirements. Section 4 is for supporting information.

2. General Description

This section provides a description of the movie ticketing system and its operating environment. It mainly summarizes the functions of our website, the user base that uses the website, the limitations of the website, and the assumptions and dependencies in the design of the website. This section provides information for the third section, but does not offer detailed design and implementation plans.

2.1 Product Perspective

The cinema ticketing system is an independent system that runs on the website. Users can access our website by searching through the browser or by entering the website address. No software installation is required. The system supports both online and in-person purchasing, with seat and showtime data updated live and shared between the website and in-person point of sale. The website can integrate external payment services such as Apple Pay and PayPal. Users can register as our users by using their Apple account or Google account.

2.2 Product Functions

For users of the website: Create an account and log in, view movie information, select the screening time, choose the viewing seat, purchase movie tickets with regular, deluxe, student, military, or senior pricing, purchase or use a membership card to reserve tickets, view movie tickets and viewing records, submit post-purchase feedback, and view movie ratings and critic reviews.

For cinema managers and customer needs: Add or update movie information, create or modify screening times, manage screening halls and seats, update movie ticket prices, view sales and reservations, and the function for canceling screenings due to special circumstances.
For the backend maintenance personnel: manage user accounts, control employee access rights, maintain website information, and monitor website operation status.

2.3 User Characteristics

Our website can be divided into three types of users: customers, cinema staff and back-end maintenance personnel.

Moviegoers are the main users of this system. They use the website to access their accounts, view movie and screening information, select seats and purchase movie tickets. Since most

moviegoers only use the website when planning to watch a movie, they should be occasional users with varying levels of technical experience. They should be able to complete ordinary tasks with little or no formal training. For new users, brief instructions are sufficient. When introducing new features, simple guidance can be provided. Moviegoers can only use the customer-oriented functions.

Cinema management staff will frequently use this system in their daily operations to update movie information, manage seating arrangements, schedule showtimes, and allocate movies to screening halls. These users should possess basic computer skills and knowledge of theater operations. New staff may require initial training and guidance from experienced staff before independently using the management functions. Theater management has limited administrative privileges, allowing them to manage designated website content and operational information, but they do not have full system administrator privileges.

System administrators and maintenance personnel should possess high-level technical skills and have received specialized training. They are responsible for managing user and employee accounts, configuring system settings, monitoring system operation, diagnosing and resolving technical issues. They have the highest level of management authority over the website; therefore, this role must be restricted to authorized personnel.

2.4 General Constraints

There are the following points in regards to the limitations of this system. The limitations of the platform, as the system needs to run through a browser, may involve issues related to browser compatibility. External platform restrictions: Due to the need to comply with the third-party payment service interface and usage rules, if the third-party service is unavailable, then both payment and login functions will not be able to operate. At present, in terms of permissions, different restrictions are imposed on different user groups. Security and privacy restrictions mean that user data cannot be accessed by unauthorized individuals. The system must comply with applicable data privacy and payment security regulations. The project has a budget of $500,000 and a 4-month timeline to deliver a viable prototype, and covers 20 theaters in the San Diego area (Pacific time zone), supporting at least 1,000 concurrent users.

2.5 Assumptions and Dependencies

Assumptions
The system assumes that the user has a stable internet connection and accesses the website through a supported web browser. It also assumes that the user provides accurate and valid account information, and that the customer has at least one system-supported payment method. The system assumes regular theaters have 150 seats and deluxe theaters have 75 seats, and that ticket refunds are handled in person by theater staff rather than through the system.

Dependencies

The website relies on third-party payment services such as Apple Pay and PayPal, as well as servers and databases, Apple and Google's third-party login services, notification services via email or text messages, and information data provided by cinema staff regarding movie showtimes, seats, and prices.

3. Specific Requirements

This will be the largest and most important section of the SRS. The customer requirements will be embodied within Section 2, but this section will give the D-requirements that are used to guide the project's software design, implementation, and testing.

3.1 External Interface Requirements

3.1.1 User Interfaces

The system shall provide a web-based graphical user interface accessible via standard web browsers (Chrome, Firefox, Safari, Edge). The interface shall include pages for browsing movies, viewing showtimes, selecting seats, checking out, and managing user accounts. The interface shall be responsive and support both desktop and mobile screen sizes. Admin users shall have access to a separate dashboard interface for managing movies, showtimes, and users.

3.1.2 Hardware Interfaces

The system shall not require specialized hardware. Users shall access the system through standard devices (desktop computers, laptops, tablets, smartphones) with internet connectivity. The system shall support standard input devices (keyboard, mouse, touchscreen).

3.1.3 Software Interfaces

The system shall interface with a payment processing service to handle credit card transactions. The system shall interface with a relational database to store movie, user, and transaction data. The system may interface with an email service to send booking confirmations and notifications. The system shall interface with a third-party movie data API, such as Rotten Tomatoes or IMDB, to retrieve movie information such as posters, descriptions, ratings, and critic reviews.

3.1.4 Communications Interfaces

The system shall communicate over HTTPS to ensure secure data transmission between the client and server. The system shall use standard internet protocols

(TCP/IP) for all client-server communication. The system shall send email notifications over SMTP for booking confirmations, cancellations, and password resets.

3.2 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.2.1 Employee operation

3.2.1.1 Introduction:
The authorized cinema staff are required to update the information, prices and showtimes of the movies. The modified version can be seen by customers, but this function cannot be accessed by customers.

3.2.1.2 Inputs

The following inputs are required:

- Employee login credentials, including the employee account and password
- Movie title
- Movie synopsis
- Movie genre
- Movie rating
- Movie poster
- Showtime
- Auditorium or screening location
- Ticket price

3.2.1.3 Processing
After the system is received, it is necessary to check whether the employee account is correct. Based on the movie information, make the necessary modifications. Check whether the new information is complete and the format is correct. Save the modified information to the database, update the information so that customers can see it, and record the modification time and the personnel information.

3.2.1.4 Outputs

The system shall produce the following outputs:

- A confirmation message indicating that the movie information was successfully updated
- The updated movie information displayed on the employee management page
- The updated movie information displayed on the customer-facing page

- A modification record containing the changes, time of modification, and responsible employee, accessible only to authorized staff

3.2.1.5 Error Handling
Employee account error, movie information error, network or database connection failure. Inform the employees when an error occurs.

3.2.2 Purchase Tickets

3.2.2.1 Introduction

This feature allows a customer to browse movies, select a showtime, choose seats, and complete payment to purchase tickets online. Pricing varies by ticket type (regular, deluxe, student, military, senior), and seating is either assigned (deluxe theaters) or general admission (regular theaters). Tickets may be purchased between two weeks in advance and up to 10 minutes after the showtime starts, with a limit of 20 tickets per transaction. Unfinished purchases are abandoned and seats released after 5 minutes.

3.2.2.2 Inputs

Movie, showtime, and seat selection
Ticket type and quantity
Payment information or membership card
Customer account information (if logged in)

3.2.2.3 Processing

The system shall verify the showtime falls within the allowed booking window, check and reserve seat availability for 5 minutes during checkout, and calculate total cost based on ticket type and quantity. It shall process payment through the payment gateway or apply a valid membership card, then release the reservation if payment is not completed within the 5-minute window.

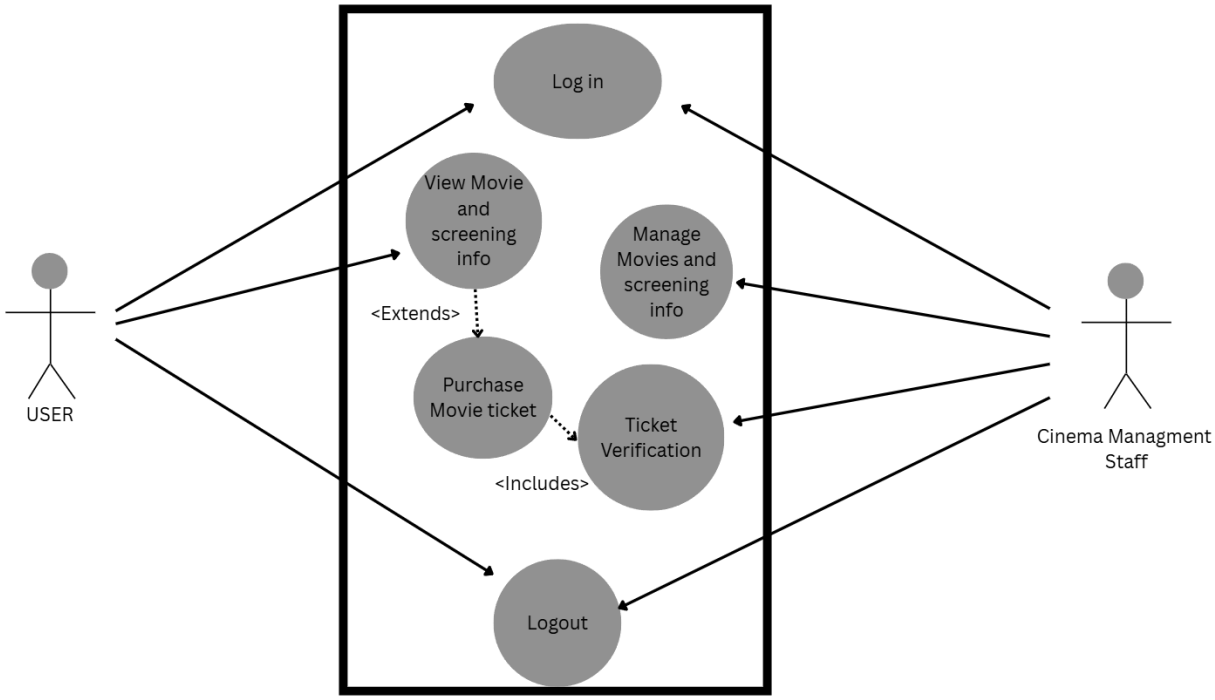
3.2.2.4 Outputs

On-screen and emailed order confirmation (movie, showtime, seats, ticket type, total cost, order number)
Updated seat availability
Post-purchase feedback prompt

3.2.2.5 Error Handling

The system shall reject purchases outside the booking window, orders exceeding 20 tickets, and failed payments, prompting the customer to correct the issue. If selected seats become unavailable during checkout, or the 5-minute window expires, the system shall release the reservation and notify the customer.

3.3 Use Cases



3.3.1 Use Case #1: Log In

Actor: User (Customer or Cinema Management Staff)
Description: A user logs into the system to access account-specific features.

Preconditions: The user has a registered account.
Main Flow:

1. User navigates to the login page.
2. User enters login credentials.
3. System verifies the credentials.
4. System grants access to the appropriate interface (customer-facing pages for a customer, employee management page for cinema staff).
   Postconditions: The user is logged in and has access to role-appropriate features.

3.3.2 Use Case #2 Purchase Movie Ticket

Actor: User (Customer)
Description: A customer views movie and screening information, selects a showtime, chooses seats, and completes payment to purchase tickets, which are then verified at the theater.
Preconditions: The customer has viewed available movies and showtimes.
Main Flow:

1. Customer selects a showtime and desired seats (assigned seating for deluxe theaters, general admission for regular theaters).
2. Customer selects ticket type and quantity (regular, deluxe, student, military, senior).
3. Customer enters payment information or applies a membership card.
4. System verifies the booking window, reserves the selected seats, and processes payment.
5. System confirms the purchase and displays/emails the order confirmation.
6. At the theater, the customer's ticket is verified against their booking record before entry.
   Postconditions: Tickets are purchased, seats are reserved, a confirmation is sent to the customer, and the ticket is verified for entry.

3.3.3 Use Case #3: Manage Movies and Screening Info

Actor: Cinema Management Staff
Description: Authorized cinema staff add, update, or remove movie and showtime information.
Preconditions: The staff member is logged into the employee management page.
Main Flow:

1. Staff member selects a movie to add, update, or remove.
2. Staff member enters or modifies movie details, showtimes, theater assignment, and ticket price.
3. System validates the input and saves the changes.
4. System updates the information on both the employee management page and the customer-facing page.
   Postconditions: Movie and screening information is updated and visible to customers.

3.4 Classes / Objects

3.4.1 Class / Object #1 Movie

3.4.1.1 Attributes

- movieID (Integer): unique identifier for the movie
- title (String): name of the movie
- description (String): short synopsis
- genre (String): movie genre
- rating (String): MPAA rating (e.g., PG-13)
- duration (Integer): running time in minutes
- releaseDate (Date): date the movie is released
- posterURL (String): link to the poster image
- showtimes (List): scheduled screenings, each with theater, date, time, price, and seats available

3.4.1.2 Functions

- getMovieDetails(): returns the movie's information for display
- getShowtimes(): returns available showtimes for the movie
- checkSeatAvailability(showtime): returns the number of open seats for a showtime
- addMovie(), updateMovie(), removeMovie(): admin-only functions for managing the movie catalog

3.4.2 Class / Object #2 Users

3.4.2.1 Attributes

- userID (Integer): unique identifier for the user
- name (String): user's full name
- email (String): email address, used for login
- password (String): stored as a hashed value
- role (Enum: Customer, Admin): determines user permissions
- paymentInfo (Object): saved payment method (Customer only)
- purchaseHistory (List): past ticket purchases (Customer only)
- membershipStatus (Boolean): indicates whether the user holds a paid membership for ticket reservations

3.4.2.2 Functions

All users:

- register(), login(), logout()
- updateProfile()

Customer:

- searchMovies(): find movies by title, genre, or date
- selectSeats(showtime): choose seats for a screening
- purchaseTickets(): pay for and confirm a booking
- cancelBooking(): cancel a purchase where allowed
- viewPurchaseHistory(): view past and upcoming tickets

Admin:

- addMovie(), updateMovie(), removeMovie(): manage the movie catalog
- manageShowtimes(): add or change screenings and prices
- manageUsers(): view or deactivate customer accounts
- overridePurchase(): corrects or reverses a customer purchase error
- viewSalesReports(): view ticket sales data

3.5 Non-Functional Requirements

Non-functional requirements may exist for the following attributes. Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transaction shall be processed in less than a second, system downtime may not exceed 1 minute per day, > 30 day MTBF value, etc).

3.5.1 Performance

The system shall respond to user actions such as searching movies, selecting seats, and completing checkout within a time frame that does not disrupt the user experience. Specific target response times (e.g., page load time, transaction processing time) are to be determined based on team decision.

3.5.2 Reliability

The system shall prevent double booking of the same seat for the same showtime under concurrent access. A specific mean time between failures value is to be determined, as this project does not include a live deployment against which reliability could be measured.

3.5.3 Availability

As this system is not deployed to production, an uptime percentage is not applicable. If the system were deployed, an availability target (e.g., percentage uptime, maximum downtime per month) would need to be defined at that time.

3.5.4 Security

The system shall not store user passwords in plain text. The system shall not store complete payment card information directly; payment processing would rely on a third-party processor in a real implementation. Access to admin functions shall be restricted to authenticated users with the Admin role.

3.5.5 Maintainability

The system shall be organized into separate components for movie management, user accounts, and ticket purchasing, so that each part of the system could be modified independently. Coding standards and documentation practices are to be determined based on team decision.

3.5.6 Portability

The system's interface shall be designed to function on standard modern web browsers and to support both desktop and mobile screen sizes. Specific browser and version support and deployment environment are to be determined, as no deployment platform has been selected.

3.6 Inverse Requirements

State any *useful* inverse requirements.

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS’s requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

6. Software Design Specification

This section is the Software Design Specification for GMT (Generic Movie Tickets). It continues the requirements in Sections 1–5 and is written for the developers who will implement and later maintain the system.

6.1 System Description

GMT (Generic Movie Tickets) is a web-based movie ticketing system designed to support ticket sales and theater management for 20 theaters in the San Diego area.

Customers can create and manage their accounts, browse movie descriptions and screening times, select cinemas and seats, purchase movie tickets through supported payment services, and receive electronic reservation confirmations.

The cinema staff log in to the management page using their employee accounts and passwords, add or modify movie information, manage the details of the movie details interface, set or modify the screening time, manage the screening venue and seats, modify the movie ticket prices, view the modification records of movie information, and ensure that all relevant information about the movies is accurate.

System maintenance personnel need to manage customer and employee accounts, set different account permissions, monitor the website, servers and databases, modify necessary system configurations, identify and fix system errors, handle technical issues, review logs and abnormal records, and back up and restore data.

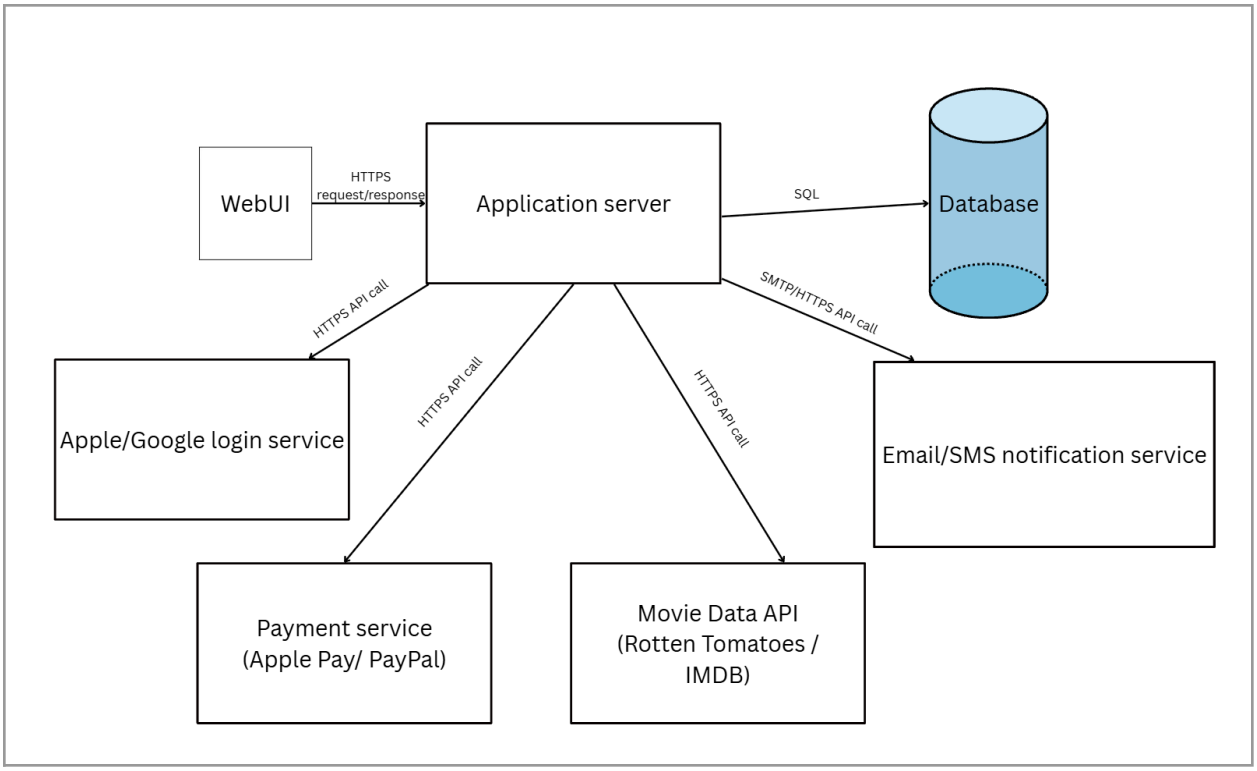
External Services.

- Apple Pay and PayPal: Handling Online Payments;
- Apple and Google Login: Facilitates customer registration or login;
- Email/SMTP and SMS services: Sending ticket purchase confirmations, electronic tickets and password reset notifications;

6.2 Software Architecture Overview

6.2.1 Software Architecture (SWA) Diagram

[Insert the architectural diagram. Show all major components AND the connectors between them, and label each connector with its interaction type. Components suggested by Section 3.1: Web UI (customer and admin dashboard), application server, relational database, payment service (Apple Pay / PayPal), Apple/Google login, email service (SMTP), third-party movie data API.]



6.2.2 SWA Description

Components

Component	Responsibility	Interacts With
WebUI	Browser-based interface for customers (browsing, booking) and staff (movie/showtime management, admin dashboard)	Application server
Application server	Hosts business logic, including Movie and User classes; processes requests and enforces business rules	Web UI, Database, Payment service, Login service, Notification service, Movie data API
Database	Stores movie, user, showtime, and transaction data, partitioned by theater	Application server
Payment service (Apple Pay / PayPal)	External service that processes customer payment transactions	Application server
Apple/Google login service	External service used for account registration and authentication	Application server

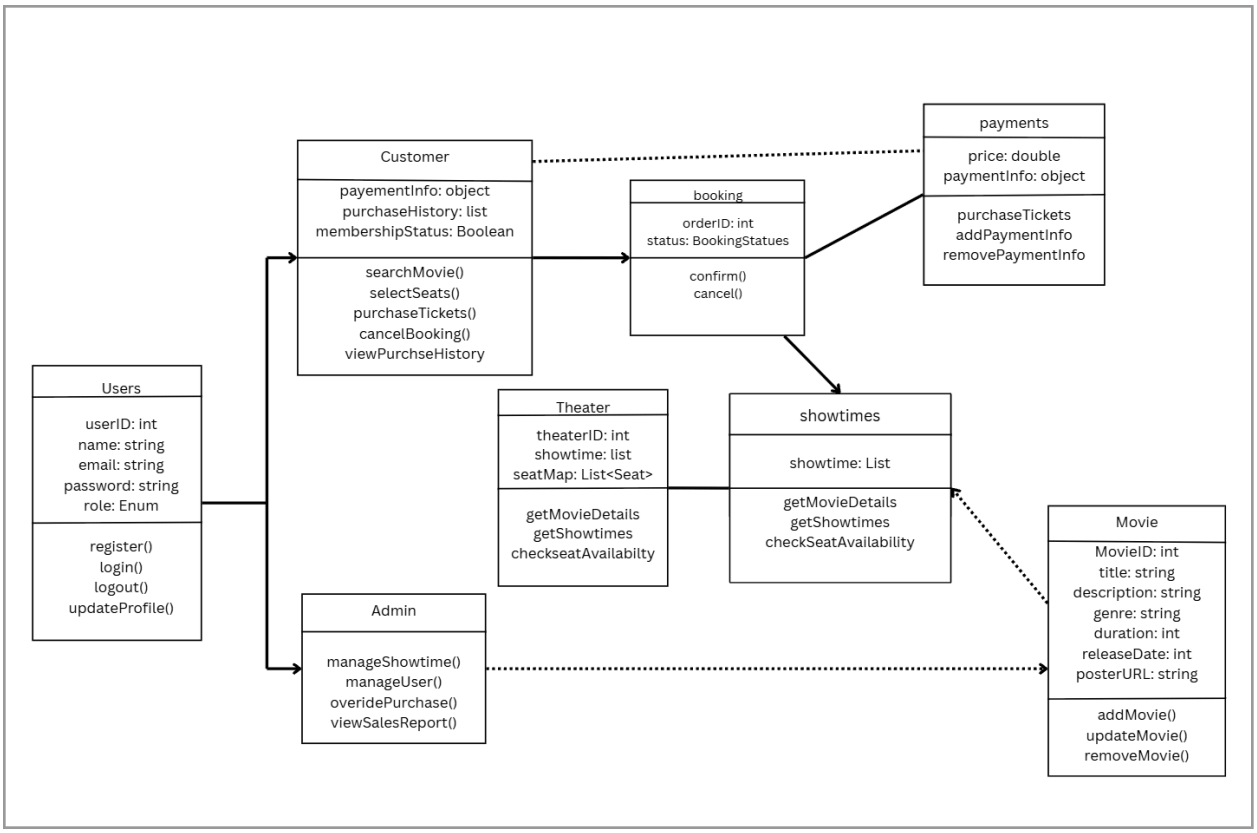
Component	Responsibility	Interacts With
Email/SMS notification service	External service used to send booking confirmations and notifications	Application server
Movie data API (Rotten Tomatoes / IMDB)	External service providing movie ratings, descriptions, and critic reviews	Application server

Connectors

Connector (From -> To)	Type / Protocol	Data Exchanged
Web UI -> Application server	HTTPS request/response	User actions (browse, select seats, purchase, login) and page data
Application server -> Database	SQL (transactions)	Movie, user, showtime, and booking records
Application server -> Payment service	HTTPS API call	Payment authorization and confirmation
Application server -> Login service	HTTPS API call(OAuth)	Authentication tokens, account verification
Application server -> Notification service	SMTP / HTTPS API call	Booking confirmations, cancellations, password resets
Application server -> Movie data API	HTTPS API call	Movie ratings, descriptions, critic reviews

6.2.3 UML Class Diagram

[Insert the complete UML class diagram: every class with name, attributes, operations, and the connections between classes (association, aggregation, composition, inheritance) with multiplicities. Section 3.4 defines Movie and Users; add the other classes your design needs (e.g., Showtime, Theater/Auditorium, Seat, Booking/Ticket, Payment, Membership).]



6.2.4 Class Descriptions

6.2.4.1 Class: Movie

Overview

Represents a movie in the catalog and the screenings scheduled for it. Admin-only operations manage the catalog.

Attributes

Attribute	Datatype	Description
movieID	Integer	unique identifier for the movie
title	String	name of the movie
description	String	short synopsis
genre	String	movie genre
rating	String	MPAA rating (e.g., PG-13)
duration	Integer	running time in minutes
releaseDate	Date	date the movie is released

Attribute	Datatype	Description
posterURL	String	link to the poster image
showtimes	List	scheduled screenings, each with theater, date, time, price, and seats available

Operations

Operation	Parameters (name: type)	Returns	Description
getMovieDetails	none	Movie	returns the movie's information for display
getShowtimes	none	List	returns available showtimes for the movie
checkSeatAvailability	showtime: Showtime	Integer	returns the number of open seats for a showtime
addMovie / updateMovie / removeMovie	movie: Movie (add/update), movieID: Integer (remove)	Boolean	admin-only functions for managing the movie catalog

6.2.4.2 Class: Users

Overview

Represents any person with an account. The role attribute (Customer or Admin) determines which operations are permitted.

Attributes

Attribute	Datatype	Description
userID	Integer	unique identifier for the user
name	String	user's full name
email	String	email address, used for login
password	String	stored as a hashed value
role	Enum (Customer, Admin)	determines user permissions
paymentInfo	Object	saved payment method (Customer only)
purchaseHistory	List	past ticket purchases (Customer only)

Attribute	Datatype	Description
membershipStatus	Boolean	whether the user holds a paid membership for ticket reservations

Operations

Operation	Parameters (name: type)	Returns	Description
register/login/logout	name: String, email: String, password: String (register); email: String, password: String (login)	Boolean	all users
updateProfile	userID: Integer, fields: Object	Boolean	all users
searchMovies	query: String	List	Customer: find movies
selectSeats	showtime: Showtime	List<Seat>	Customer: choose seats for a screening
purchaseTickets	showtimeID: Integer, seats: List<Seat>, ticketType: String	Boolean	Customer: pay for and confirm a booking
cancelBooking	orderID: Integer	Boolean	Customer: cancel a purchase where allowed
viewPurchaseHistory	none	List	Customer: view past and upcoming tickets
manageShowtimes	movieID: Integer, showtime: Showtime	Boolean	Admin: add or change screenings and prices
manageUsers	userID: Integer	Boolean	Admin: view or deactivate customer accounts
overridePurchase	orderID: Integer	Boolean	Admin: correct or reverse a customer purchase error
viewSalesReports	none	List<Report>	Admin: view ticket sales data

6.2.5 Data Structures

Data Structure	Type	Purpose	Used In
Showtime record	Struct (within Movie.showtimes)	Represents one scheduled screening: theaterID, date, time, price, seatsAvailable	Movie
Seat map	List<Seat>	Tracks individual assigned seats for deluxe theaters, or a simple available count for general admission	Movie, User
Ticket and purchases	Struct (within User.purchaseHistory)	Represents one booking: orderID, showtimeID, seats, ticketType, totalCost, purchaseTimestamp	User

6.3 Development Plan and Timeline

6.3.1 Partitioning of Tasks

#	Task	Description	Depends On
1	Requirement analysis and framework design	Define database setup, finalize class design, confirm architecture.	NONE
2	UI/UX Design	Design Wireframe and design customer-facing pages (movie browsing, seat selection, checkout) and admin dashboard, per 3.1.1	Task 1
3	Database design and setup	Design and implement the partitioned relational database schema for movies, users, showtimes, and tickets.	Task 1
4	Frontend Development	Build customer and admin website pages and components from the UI/UX designs	Task 2
5	User login authentication and account management	Connect to a third-party login system (Google/Apple) and encrypt the user passwords.	Task 3
6	Movie catalog and showtime management	Implement Movie class operations and admin movie/showtime management interface	Task 3,Task 4

#	Task	Description	Depends On
7	Ticket purchasing flow	Implement seat selection, checkout, booking window/timeout logic, payment gateway integration	Task 4, Task 5, Task 6
8	Testing and quality assurance	Unit and integration testing, including concurrent-access testing for seat booking	Task 7

6.3.2 Timeline

[Estimated schedule per task, with who is responsible. The project has a 4-month timeline (Section 2.4).]

Task #	Start Date	End Date	Milestone / Deliverable	Responsible
1	9/1/2026	9/24/2026	Requirement Specification & Architecture Draft	All Members
2	9/25/2026	10/8/2026	Software Design Specification (SDS) Completed	All Members
3	10/9/2026	10/25/2026	Partitioned Relational Database Schema Setup	All Members
4	10/9/2026	10/30/2026	Customer & Admin Web Layouts and Components	All Members
5	10/26/2026	11/13/2026	Third-Party Auth & Password Encryption Module	All Members
6	10/31/2026	11/18/2026	Movie Catalog & Admin Showtime Endpoints	All Members
7	11/19/2026	12/05/2026	Integrated Ticket Purchasing & Seat Lock Flow	All Members
8	12/06/2026	12/16/2026	Unit/Integration Testing & QA Test Report	All Members

6.3.3 Team Member Responsibilities

Team Member	Role	Responsibilities / Components Owned	GitHub Username
Andres Akerlundh	UI/UX Designer	responsible for the UI/UX design	AA6304

Team Member	Role	Responsibilities / Components Owned	GitHub Username
Madison Stevens	Programmer	responsible for html code and implementation of data structures	MadisonStevens98
Haoze Li	Project Manager	responsible for managing timeline, labor, and quality	hli1614

7. Test Plan

7.1 Test Objectives and Scope

This section covers the testing objectives, the functions to be tested, Unit, Functional/Integration and System tests, as well as test data, error inputs and failure scenarios.

7.1.1 Testing objective

This test plan is designed to verify whether the cinema ticketing system is operating in accordance with the requirements specified in the documentation, and whether it meets the needs of customers, staff, and system maintenance personnel. By testing individual components, the connections between components and the operation of the entire system, potential problems can be identified. The test will also verify whether the system can handle invalid inputs or unexpected malfunctions.

7.1.2 Testing which function

We will test the following functions:

- User registration, login, and account verification
- Browse movies, cinemas and available times
- Ticket price verification and display
- The availability of seats and the selection of seats
- Reserving movie tickets and payment processing
- Confirm the updated reservation information and the availability of seats on the website
- Authorization of staff login and information verification.
- Authorize the staff to manage the films, showtimes, screening halls, seats, and ticket prices.
- Account permission settings and tests to prevent unauthorized external access
- Error handling when inputting invalid information or encountering unavailable external services
- The accuracy of the system log information recorded by the staff and system maintenance personnel when operating the system
- ……

7.1.3 Unit, Functional/Integration and System testing

The test will be conducted at three levels: unit, functional/integrative, and system. Unit tests will evaluate the implementation of individual functions, such as login, payment, fare verification and logging, etc. The functional and integration tests will be a combination of unit tests. For instance, after logging in, the browsing of the target movie, booking seats, paying for the movie ticket, and confirming the reservation information - will this series of operations run smoothly? The system test will thoroughly examine the workflow from the perspectives of different users, such as the complete operation of an employee modifying movie information.

7.1.4 Test data, erroneous input and failure scenarios

This section of the test involves deliberately inputting some incorrect data to observe how the system responds to these data. It checks whether the system can correctly output these errors, such as negative movie ticket prices, or situations where images cannot be uploaded due to incorrect specifications, or cases where the system fails to save properly, etc.

7.2 Test Cases and Coverage

This section presents 10 test cases in Excel, (testCasesSample.xlsx), explaining which functions and testing levels these cases. The level of each, the test vectors used, and the parts of the design each one targets.

The three levels follow Section 7.1.3. A unit test checks one function in isolation, with its dependencies replaced by stubs. A functional (integration) test combines several units, such as the checkout page, purchaseTickets() and the payment service. A system test runs a complete workflow through the whole running system from the point of view of one type of user. The test case ID prefix (UNIT_, FUNC_, SYS_) shows the level in the Excel file.

7.2.1 Test Case Summary

Test ID	Level	Component under test	Test vectors (inputs)	What it covers (SRS trace)
UNIT_LOGIN	Unit	Users.login()	Valid credentials, authentication service stubbed to approve	Successful authentication (3.3.1)
UNIT_PAY	Unit	Customer.purchaseTickets()	Valid booking, payment service stubbed to approve	Booking confirmed after approved payment (3.2.2)

UNIT_SEAT	Unit	Movie.checkSeatAvailability()	Deluxe showtime of 75 seats: 5 reserved, then all reserved	Correct count and sold-out boundary of 0 (3.4.1, 2.5)
UNIT_PRICE	Unit	Admin.manageShowtimes()	Ticket price -5.00 (invalid), then 12.50 (valid)	Erroneous input rejected, valid input saved (3.2.1, 7.1.4)
FUNC_LOGIN	Functional	Web UI + Users.login() + Google login service	Valid Google account	Login through a third-party service (3.3.1, 2.1)
FUNC_PAY	Functional	Checkout page + purchaseTickets() + payment service	Valid test payment details, supported payment method	Payment, confirmed booking, seats unavailable, purchase history (3.2.2)
FUNC_ERR	Functional	Checkout page + purchaseTickets() + payment service	Declined payment details, then valid payment details	Payment failure message and successful retry (3.2.2.5)
SYS_BOOK	System	Whole system: customer purchase	Student ticket, 2 deluxe seats, payment sandbox, second session as in-person terminal	Full customer workflow, confirmation email, live seat data shared across channels (3.3.2, 2.1)
SYS_STAFF	System	Whole system: employee management	New showtime and price, then a customer account opening the management page	Staff workflow, customer view updated, modification record, access control (3.2.1, 3.3.3, 3.5.4)

SYS_CONC	System	Whole system: concurrent booking and timeout	Two customers pay for seat D5 at once; one customer idle for 5 minutes on seat E1	No double booking and 5-minute abandonment (3.5.2, 3.2.2)

The test sets deliberately include valid input (approved payment, valid Google account, valid price), invalid input (negative price, declined payment, unauthorized access), boundary values (a sold-out showtime, the 5-minute checkout limit) and a concurrency scenario (two customers selecting the same seat).

7.2.2 Test Targets and Scope

Figure 7.1 (next page) marks which test cases target each part of the design. It uses the class diagram from Section 6.2.3, so the tests are traced to the same Users, Customer, Admin, Movie, Showtime, Seat, Booking and Payment classes that are described in Section 6.2.4. Unit tests (green) target single operations of one class, with the database, payment service and login service stubbed. Functional tests (yellow) target the interaction between classes and the external login and payment services. System tests (orange) cover the whole system boundary, including the database, the email notification service and the movie data service, which are exercised only through the system tests. Classes shown in grey (MembershipCard) are not covered by the 10 sample cases.

Figure 7.1 reuses the components and connectors of the Software Architecture in Section 6.2 and marks which tests target each part. Unit tests (blue) target single operations of the User and Movie classes. Functional tests (green) target the connectors between the Web UI, the application server and the external login and payment services. System tests (purple) cover the whole system boundary, including the database and the notification service, which are exercised

only through the system tests.

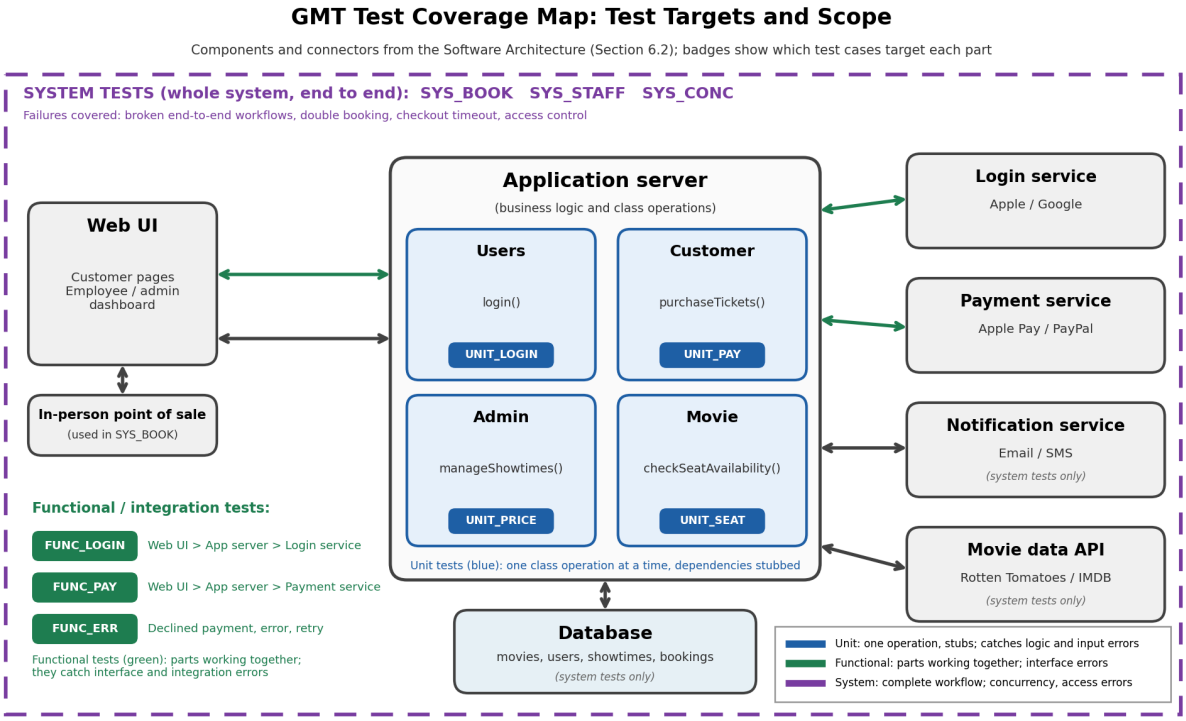


Figure 7.1: Test coverage map

7.2.3 Coverage of the Functions Listed in 7.1.2

Function listed in 7.1.2	Test cases that cover it
User login and account verification	UNIT_LOGIN, FUNC_LOGIN
Browsing movies, theaters and showtimes	SYS_BOOK
Ticket price verification and display	UNIT_PRICE, SYS_BOOK, SYS_STAFF
Seat availability and seat selection	UNIT_SEAT, SYS_BOOK, SYS_CONC
Reserving tickets and payment processing	UNIT_PAY, FUNC_PAY, FUNC_ERR, SYS_BOOK
Updated reservation information and seat availability on the website	FUNC_PAY, SYS_BOOK, SYS_CONC
Staff login and management of movies, showtimes and prices	UNIT_PRICE, SYS_STAFF

Account permissions and blocking unauthorized access	SYS_STAFF
Error handling for invalid input and unavailable services	UNIT_PRICE, FUNC_ERR, SYS_CONC

The 10 sample test cases do not cover every item in 7.1.2 (user registration and the system logs kept for maintenance personnel are not covered). A full test suite would add cases for them at the unit and functional levels.

7.3 GitHub Link

https://github.com/MadisonStevens98/CS250-04-Fall2026/tree/8ce3fd59322fa7164c7ec2d3bea718dcc51bdf4d/SDSFolder

8. Architecture Design with Data Management

8.1 Introduction

This section updates the Software Design Specification (Section 6) with an architecture diagram that includes the databases, and describes the data management strategy for GMT (Generic Movie Tickets). It explains how the design changed from the initial design in Assignment 2, which kind of database we chose and why, how many databases there are and how the data is split up, how sensitive data is protected, and which alternatives we considered and rejected.

The strategy is based on the client interview (a single database for all theaters, partitioned by theater; online and in-person sales sharing live data; 20 theaters; at least 1,000 concurrent users) and on the requirements in Section 3 (3.2.2 purchase rules, 3.5.2 no double booking, 3.5.4 security).

8.2 Updated Software Architecture Diagram

Figure 8.1 (next page) shows the updated architecture. It keeps the components of the Assignment 2 diagram (Web UI, application server, database, payment service, login service, notification service and movie data API) and adds the detail needed for data management: the application server modules, the single partitioned primary database with its five schemas, a read replica for reports, encrypted backup storage, and the in-person point of sale

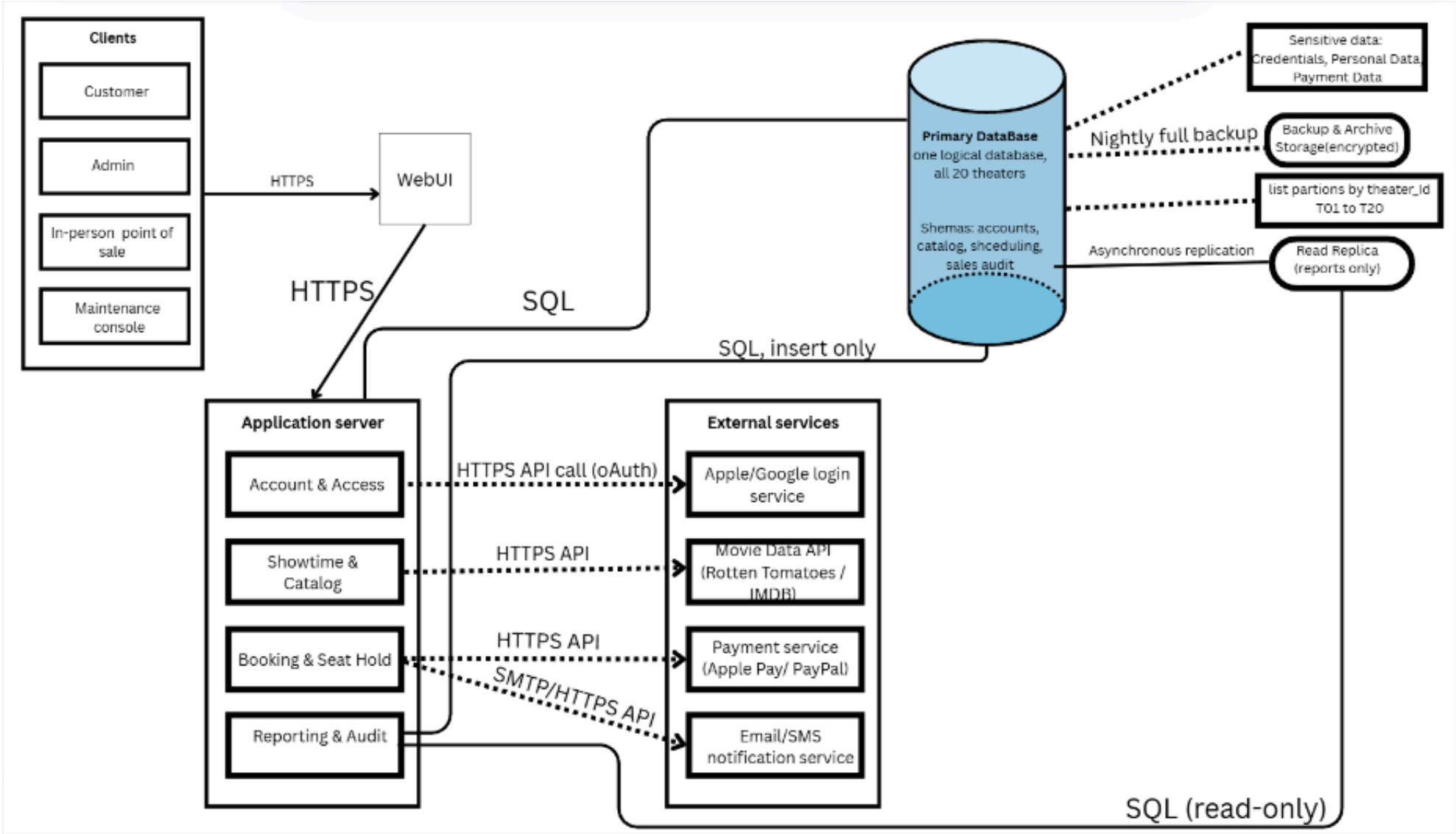


Figure 8.1: Updated software architecture diagram including the data

8.2.1 Data Tier and Application Server Modules

Component	Responsibility	Interacts with
Primary database	The single source of truth for all 20 theaters. SQL database that accepts all reads and writes. Organized into five schemas (accounts, catalog, scheduling, sales, audit); the high-volume tables are partitioned by theater (8.4.3).	Application server modules, read replica, backup storage
Partitions (T01 to T20)	One list partition per theater for the showtime, seat, booking, ticket and payment tables. A query for one theater only touches that theater's partition.	Primary database
Read replica	Read-only copy of the primary, updated asynchronously. Used for sales reports so that reports never slow down ticket sales.	Reporting & Audit module, primary database
Backup & archive storage	Encrypted nightly backups and transaction log copies, used by maintenance personnel to restore data (6.1).	Primary database
Account & Access module	Registration, login, roles and permissions. The only module that reads or writes password hashes.	accounts schema, login service
Catalog & Showtime module	Movies, theaters, showtimes and ticket prices (staff only). Stores movie scores fetched from the movie data API.	catalog and scheduling schemas, movie data API
Booking & Seat Hold module	Seat selection, the 5-minute hold, ticket pricing, payment, confirmation, and feedback. Implements the transactions in 8.4.4.	scheduling and sales schemas, payment service, email/SMS service
Reporting & Audit module	Sales reports (tickets sold per movie, cost per ticket) and the modification records of staff changes.	Read replica, audit schema
In-person point of sale	Used by theater staff to sell tickets at the theater through the same Web UI and application server, so online and in-person sales always see the same live seat data.	Web UI

8.2.2 Data-Related Connectors

Connector (From → To)	Type / protocol	Data exchanged
Application server modules → Primary database	SQL over a TLS connection; transactions	All application data. The Booking module uses row locks on seats (8.4.4).
Reporting & Audit module → Read replica	SQL, read-only	Sales and schedule data for reports
Reporting & Audit module → Primary database	SQL, insert only	Modification records and audit events
Primary database → Read replica	Asynchronous replication	Copy of every committed change
Primary database → Backup storage	Nightly full backup and log shipping every 5 minutes	Encrypted snapshots and transaction logs
Booking module → Payment service	HTTPS API call	Amount and payment method in; approval and processor token out (no card numbers)
Account module → Login service	HTTPS API call (OAuth)	Authentication tokens, account verification
Booking module → Email/SMS service	SMTP / HTTPS API call	Order confirmations, e-tickets
Catalog module → Movie data API	HTTPS API call	Movie scores and critic quotes, cached in the catalog schema

8.3 Changes from the Initial Design (Assignment 2)

The table compares the Assignment 2 design (Section 6.2) with the updated design and gives the reason for each change.

#	Item	Assignment 2 (initial design)	Updated design	Reason
1	Data stores	The workflow diagram showed three separate stores (user accounts and ticket info, showtime records, seat map). The component diagram showed one generic "Database".	One logical SQL database with five schemas. The three stores become tables in it (users and tickets, showtime, seat).	The client wants one database for all theaters. One database lets seats, bookings and payments change in one transaction.
2	Partitioning	"Partitioned by theater" was mentioned only in the component table.	Showtime, seat, booking, ticket and payment are list-partitioned by theater_id (T01 to T20).	Seat maps and showtimes are always read per theater; partitions keep these queries small and make archiving easy.
3	Read replica	Not included.	Added a read replica used only by sales reports.	Admin reports (viewSalesReports) should not slow down ticket sales.
4	Backup	Mentioned only as a maintenance task in 6.1.	Added encrypted backup and archive storage with log shipping.	Maintenance personnel must be able to back up and restore data; ticket and payment records must not be lost.
5	In-person sales	Not shown.	The in-person point of sale is a client of the same Web UI and application server.	SRS 2.1 and the client require online and in person data to be the same and live.
6	Seat holds	The 5-minute rule was a requirement only.	Seat rows carry a hold-expiry time and are locked inside short transactions.	Implements the 5-minute abandonment (3.2.2) and prevents double booking (3.5.2).
7	Sensitive data	Requirements only: hashed passwords, no full card data (3.5.4).	Column-level decisions: password hash, payment token only, encryption, least-privilege database accounts (8.4.5).	Turns the security requirements into concrete database design.
8	Tables	Data structures for Showtime, Seat map and Ticket (6.2.5), plus Movie and Users.	Added tables for theater, membership card, payment, feedback, critic review and modification record (8.4.3).	Needed by SRS 3.1.3, 3.2.1.4, 3.2.2.4 and 3.4.2; supports the Theater, showtime, booking and payment classes in the Assignment 2 class diagram.
9	Application server	One box.	Four modules (Account & Access, Catalog & Showtime, Booking & Seat Hold, Reporting & Audit), each with defined database access.	Shows which module may touch which data, which supports access control and maintainability (3.5.5)
10	Movie scores	Fetched from the movie data API.	Scores and quotes are cached in the catalog schema	Browsing still works if the external API is down (2.4).

8.4 Data Management Strategy

8.4.1 Data Inventory and Sensitivity

GMT stores persistent data about people, purchases and theater operations. The table lists each kind of data, how sensitive it is, and where it lives.

Data	Examples	Sensitivity	Stored in	Main protection
Credentials	Password hash	Very high	accounts.users	Salted hash only (never plain text, 3.5.4); readable only by the Account module
Personal data	Name, email, role	High (personal)	accounts.users	TLS in transit; encryption at rest; role-based access
Payment data	Method, amount, processor token, transaction reference	High	sales.payment	Token and reference only; no card number or security code is stored (3.5.4)
Purchase data	Orders, tickets, QR codes, purchase history	Medium (personal)	sales.booking, sales.ticket	Visible only to the customer and authorized staff
Membership	Card ID, expiry, active flag	Medium	accounts.members hip_card	Same as account data
Schedule data	Theaters, showtimes, prices, seat status	Low, but integrity is critical	catalog and scheduling schemas	Only staff can change; constraints prevent invalid prices and double holds
Catalog data	Titles, posters, scores, quotes	Public	catalog schema	Integrity checks only
Audit data	Who changed what, and when	Integrity-critical	audit.modification_record	Insert-only; readable by authorized staff only (3.2.1.4)
Feedback	Rating after a purchase	Low	sales.feedback	Linked to an order, not shown with names

8.4.2 Database Technology: SQL

We chose a **relational (SQL) database**, for example PostgreSQL (MySQL would also meet the needs). The reasons come directly from the requirements:

Requirement or need	Why SQL fits
Highly related data (movie → showtime → seat → booking → ticket → payment)	Foreign keys and joins model these relationships directly and keep them consistent.
A purchase changes seats, booking and payment together	ACID transactions make the changes all succeed or all fail, so we never sell a seat without a booking or take payment for nothing.
No double booking under concurrent use (3.5.2)	Row-level locks and unique constraints let the database itself reject a second sale of the same seat.
Sales data for the client (tickets sold per movie, cost per ticket)	Grouping and totals in SQL answer these questions directly.
Invalid input must be rejected (negative prices, more than 20 tickets, duplicate email)	CHECK, NOT NULL and UNIQUE constraints enforce the rules even if the application has a bug.
Partitioning, backups, encryption and access control	Mature relational products provide all four as standard features.

8.4.3 Number of Databases and Logical Organization

Number of databases: one logical database. The client asked for one database for all theaters, partitioned by theater. We follow this. In deployment it is one primary instance plus a read replica and backups, but there is only one source of truth. This is also what makes online and in-person sales show the same live data without any synchronization between systems.

Partitioning. The showtime, seat, booking, ticket and payment tables are list-partitioned by theater_id, giving 20 partitions (T01 to T20). These tables are the largest, and almost every query on them is for one theater (the seat map of one showtime, the sales of one theater). Every such query must include theater_id so that it only touches one partition. Customers can buy tickets at any theater, so the account and catalog data is not partitioned. The theater, membership, critic review, feedback and modification tables are small and stay unpartitioned.

Logical organization. The data is grouped into five schemas by purpose. Each application module gets database permissions only for the schemas it needs (see 8.4.5).

Schema	Tables	Purpose	Partitioned tables
accounts	users, membership_card	Customer and employee accounts, roles, membership cards	None
catalog	movie, critic_review	What is showing, with scores and quotes	None
scheduling	theater, showtime, seat	When and where movies show; which seats are free, held or sold	showtime, seat
sales	booking, ticket, payment, feedback	Orders, tickets, payments and feedback	booking, ticket, payment
audit	modification_record	History of staff changes	None

Integrity rules. The database enforces these rules in addition to the application: email is unique; price is never negative; seat is unique per showtime, row and number; a booking has between 1 and 20 tickets; payment amount equals the booking total when approved; a seat can only be Held or Sold by one booking at a time; every foreign key must point to an existing row.

Figure 8.2 (next page) shows the logical data model: every table, its key columns, how the tables are related, which tables are partitioned, and which columns are sensitive.

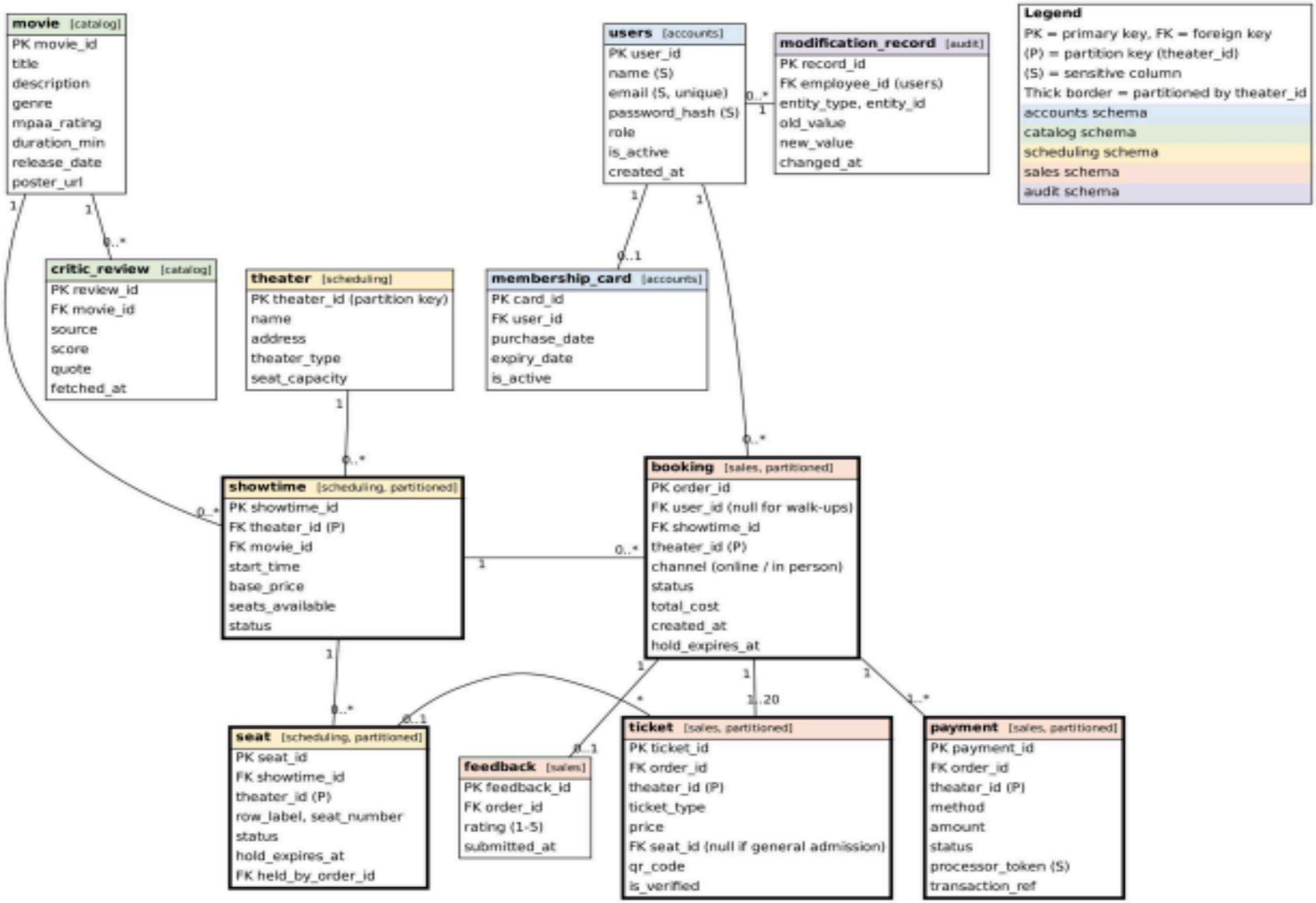


Figure 8.2: Logical data model (tables, keys, partitioning and sensitive columns)

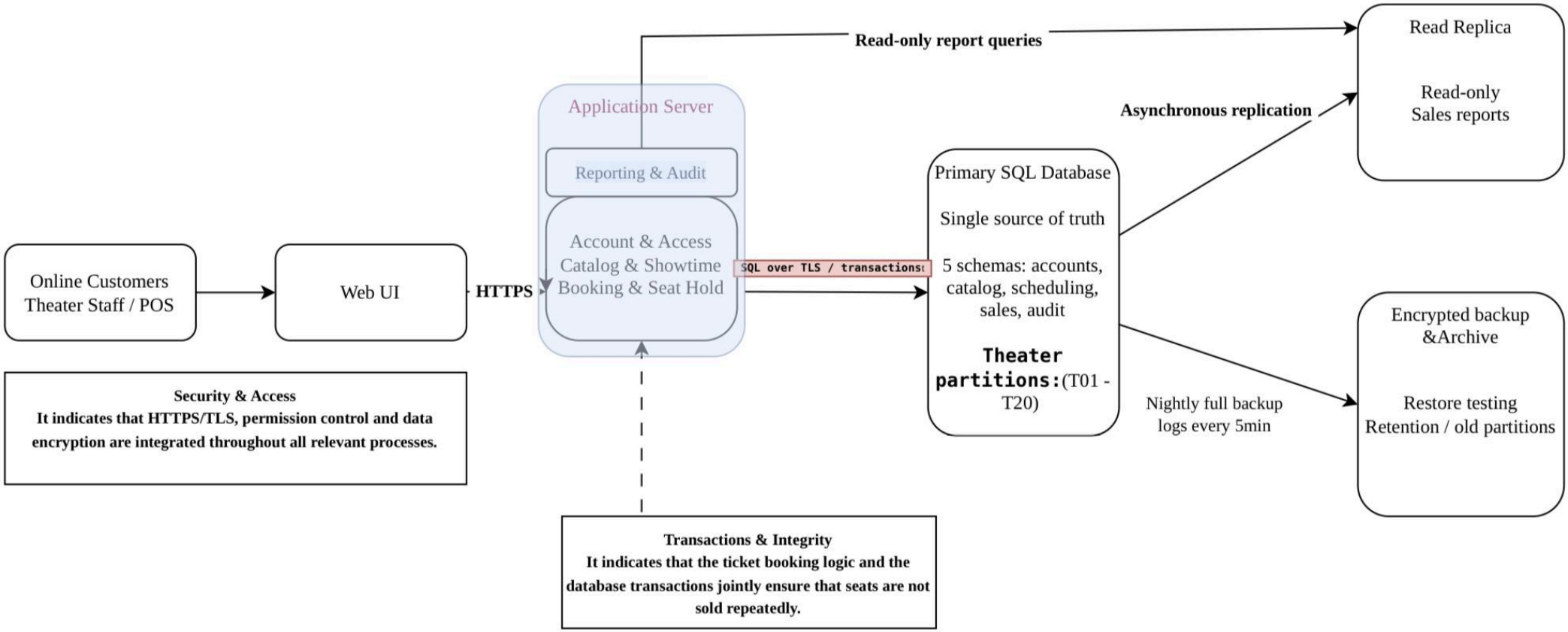


Figure 8.3 Data Management Strategy for the Movie Ticketing System

8.4.4 Transactions, Concurrency and Live Data

Seat availability is the most contested data, so it is handled inside short database transactions:

1. Selecting seats (deluxe theaters, assigned seats). In one transaction the Booking module locks the chosen seat rows, checks that all are Available, sets them to Held with an expiry time 5 minutes ahead, and creates a Pending booking. If any seat is no longer available, the transaction is rolled back and the customer is told to choose again.

2. Selecting seats (regular theaters, general admission). There are no seat rows. One statement lowers showtime.seats_available by the ticket count only if enough seats remain, so two customers cannot take the last seats.

3. Payment. The payment service is called outside the database transaction. When it approves, a second short transaction marks the seats Sold, the booking Confirmed and the payment Approved. If the payment is declined, the hold stays until it expires or the customer retries (this is the FUNC_ERR test).

4. Hold expiry. A scheduled job runs every minute and also runs when seats are read. It sets expired holds back to Available, marks the booking Abandoned, and restores seats_available. This implements the 5-minute abandonment rule (3.2.2).

Online customers and the in-person point of sale use the same module and the same tables, so both see live data. These transactions are what the SYS_CONC system test exercises.

8.4.5 Protection of Sensitive Data

Measure	How it is applied
Data in transit	All traffic between browser, application server and database uses HTTPS / TLS (3.1.4).
Data at rest	Database volumes and backups are encrypted. Name and email are stored in the users table only.
Passwords	Stored only as salted hashes (for example bcrypt or Argon2). A forgotten password is reset, never recovered (3.5.4).
Payment data	Only a processor token and transaction reference are stored. Card numbers and security codes stay with Apple Pay / PayPal, which keeps GMT out of most card-data compliance work.
Least privilege	Each module connects with its own database account. Only the Account module can read password hashes. The Reporting module reads from the replica and can only insert audit rows. No module uses an administrator account.
Application access control	Admin functions are limited to authenticated users with the Admin role (3.5.4); customers can only read their own bookings.
Safe queries	All queries are parameterized to prevent SQL injection. Logs never contain passwords, tokens or full personal data.
Audit trail	Staff changes to movies, showtimes and prices are written to an insert-only table with employee, time, old and new value.
Deactivation and deletion	Accounts are deactivated, not deleted, so orders stay valid. If an account must be removed, personal fields are replaced with anonymous values.

8.4.6 Backup, Recovery and Retention

Maintenance personnel back up and restore data (6.1). We propose: a nightly full backup, transaction logs copied every 5 minutes (so at most about 5 minutes of data could be lost), and the read replica as a second copy. Maintenance personnel test a restore regularly. Retention periods are proposals to be confirmed with the client: ticket, booking and payment records are kept for 7 years for accounting; Abandoned bookings are deleted after 30 days; modification records are kept as long as the data they describe; partitions older than the retention period are archived by detaching the partition.

8.4.7 Capacity Estimate

The estimate uses upper bounds from the client interview to check that one database is enough.

Quantity	Calculation	Result
Showtimes per day	20 theaters × 8 showtimes (client: 6 to 8)	160 per day
Seats (upper bound, all sold)	160 showtimes × 150 seats	24,000 per day, about 8.8 million per year
Size of the largest tables	8.8 million rows × about 150 bytes	about 1.3 GB per year per table, before indexes
Seats held at the same moment	1,000 users × at most 20 tickets	at most 20,000 rows
Purchase rate	1,000 users each buying within a 5- minute window	about 3 purchases per second on average; a 10× burst is about 33 per second

These numbers are small for a relational database on one server, so a single primary with a read replica meets the requirement of at least 1,000 concurrent users (2.4). Only theaters with assigned seating (75 seats) store seat rows, so the real seat count is lower than this upper bound.

8.5 Design Decisions, Alternatives and Tradeoffs

This section records the main decisions, the alternatives we considered (in both technology and organization of the data), and the tradeoffs.

8.5.1 Database Technology

Option	Advantages	Disadvantages	Decision
Relational SQL (PostgreSQL / MySQL)	ACID transactions across seats, bookings and payments. Joins and constraints. Easy sales reports. Built-in partitioning and encryption.	Scaling writes beyond one server is harder. Schema changes need migrations.	**Chosen.** The data is structured and consistency is the main risk.
Document store (for example MongoDB)	Flexible structure (useful for movie metadata). Easy to spread across servers.	Multi-document transactions are weaker and slower. Joins and reports are harder. Movie and showtime data would be duplicated.	Rejected. Double booking prevention and reports matter more than flexibility.

Key-value / in-memory store (for example Redis) for seat holds	Built-in expiry matches the 5-minute hold. Very fast.	A second system to run. Data can be lost on restart. Must be kept in step with the SQL database.	Not used now. Possible later if load grows.
SQL plus a document store for movie data	Each store used for what it does best.	Two technologies for a small team and a 4-month, $500,000 prototype.	Rejected for the prototype.

8.5.2 Number and Organization of Databases

Option	Advantages	Disadvantages	Decision
One database partitioned by theater	Matches the client request. One source of truth for online and in person sales. One transaction can cover seat, booking and payment. One system to back up and manage.	Single point of failure. Every query on partitioned tables must include theater_id. Heavy load at one theater can affect others.	**Chosen.** Mitigations are in 8.5.4.
One database per theater (20 databases)	Strong isolation. A failure affects only one theater. Easy to scale by theater.	Customer accounts must be duplicated or kept in an extra global database. Reports across theaters need data from 20 places. 20 times the backups and upgrades.	Rejected.
Separate databases by function (accounts, catalog, bookings)	Sensitive data is isolated. Each part can scale on its own.	A purchase would span several databases, which needs distributed transactions or complicated recovery logic. More servers and work.	Rejected. One database with separate schemas and permissions gives most of the isolation.
One database, no partitions	Simplest to set up.	All theaters share the same large tables and indexes. Archiving old data is harder. Cannot easily move one theater later.	Rejected. Partitioning is cheap and prepares for growth.

8.5.3 Handling of Payment and Credential Data

Option	Advantages	Disadvantages	Decision
Store processor token only	Little card-data risk. Customers can still pay quickly with the saved method.	Depends on the payment service being available (2.4).	**Chosen.**
Store full card data ourselves	Independent of the payment service.	Very high security and compliance cost and risk.	Rejected (also forbidden by 3.5.4).
Passwords as salted hashes	A database leak does not reveal passwords.	Passwords cannot be recovered, only reset.	**Chosen.**
Passwords encrypted (reversible)	Passwords can be shown or recovered.	A key leak exposes every password.	Rejected.

8.5.4 Accepted Tradeoffs

	Why	How we reduce the risk
One database is a single point of failure.	The client wants one database, and one database keeps seat, booking and payment consistent.	Read replica that can be promoted; nightly backups plus log shipping every 5 minutes; tested restores.

Writes cannot scale across many servers.	The estimate in 8.4.7 is far below what one server can handle.	The theater_id partition key makes it possible to move theaters to separate servers later.
Partition key discipline: queries must include theater_id.	Gives small, fast queries and easy archiving.	Data access code takes theater_id as a required parameter; code review and tests check it.
Reports may be a few seconds out of date.	Keeps heavy reports away from ticket sales.	Reports run on the replica; live seat data always comes from the primary.
Cached movie scores can be out of date.	The site keeps working if the movie data API is down.	Scores are refreshed on a schedule and the fetch time is stored.
Holds are released by a scheduled job.	Simple and reliable; no extra system.	The hold time is also checked whenever seats are read, so an expired hold is never treated as taken.

If load grows beyond one server, the first changes would be an in-memory store for seat holds and moving busy theaters to their own database servers using the existing theater_id partition

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2